C PROGRAMMING PROJECT REPORT ON

SLIDE TRAVERSAL AND ENERGY-BASED MOVEMENT SIMULATION

Submitted by

INIYHA .D.K.
ROLL NUMBER: 25BCS118

Under the Guidance of

Trainer Name
Technical Trainer
Department of Computer Science
KG College of Arts and Science

B.Sc. Computer Science

Academic Year: 2026 – 2027

OCTOBER 2026

DECLARATION

I hereby declare that the project entitled “SLIDE TRAVERSAL AND ENERGY-BASED MOVEMENT SIMULATION” submitted in partial fulfillment of the requirements for the course **C Programming** is a record of my work prepared for academic purposes.

I further declare that this project report has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title at this or any other institution.

I have acknowledged the references used in preparing this report wherever applicable.

October 2026
Coimbatore

_____

INIYHA .D.K.
Roll Number: 25BCS118

CERTIFICATE

This is to certify that the project entitled “SLIDE TRAVERSAL AND ENERGY-BASED MOVEMENT SIMULATION” is a project report prepared by **INIYHA .D.K. (25BCS118)**, B.Sc. Computer Science, KG College of Arts and Science, in partial fulfillment of the requirements for the course **C Programming**.

The report describes a C program that reads diagonal slide coordinates, simulates movement from a starting position, checks slide-related blocking conditions, and updates the remaining energy.

Place: Coimbatore
Date: _____

_____	_____
Guide	Head of the Department
(Technical Trainer, KGCAS)	(KGCAS)

Submitted for the Viva-Voce Examination held on _____

TABLE OF CONTENTS

S. No.	Topics	Page No.
1	Introduction	6
1.1	Objective	6
1.2	Overview	6
1.3	Features	6
2	Program Analysis	7
2.1	Problem Definition	7
2.2	Project Overview	7
2.3	Scope and Limitations	7
3	System Design	8
3.1	Modules	8
4	Methodology	9
4.1	Problem Analysis	9
4.2	System Design	9
4.3	Program Development	9
4.4	Testing and Validation	10
4.5	Result and Documentation	10
4.6	Algorithm	10
4.7	Flowchart	11
5	Source Code	12–13
6	Input and Output	14
7	Discussion and Conclusion	15
8	References	15

ABSTRACT

The project “Slide Traversal and Energy-Based Movement Simulation” is a C programming exercise that processes a set of diagonal slides and simulates movement from a given starting coordinate. Each slide is described by two endpoints. The program stores the slide coordinates in a structure array and normalizes the endpoints according to their vertical positions.

The program receives the number of slides, the coordinates of each slide, and the starting position with an energy value. It repeatedly searches for a slide that lies under the current horizontal coordinate and does not exceed the current vertical position. Depending on the selected endpoint and movement direction, the program moves along the slide while decreasing the vertical coordinate and the available energy.

A helper function checks whether another slide blocks movement at a particular coordinate and direction. If a blocking condition is detected, the program compares the remaining energy with the product of the current x and y coordinates. Finally, the program prints the ending coordinate.

The project demonstrates the use of structures, arrays, loops, conditional statements, functions, integer arithmetic, and the standard C libraries stdio.h and stdlib.h.

1. INTRODUCTION

Computer programs often represent a real-world or puzzle-like situation using coordinates, rules, and repeated decisions. This project uses C language to model a set of diagonal slides on a coordinate plane and to calculate the final position of a moving point.

Each slide is stored using its lower and upper endpoints. The program reads the starting x-coordinate, y-coordinate, and energy. It then searches for an appropriate slide below the current position and moves along that slide when the selected slide requires traversal.

The simulation also includes a blocking check. When another slide starts at the same coordinate and has the same movement direction, the helper function reports that the path is blocked. The program applies an energy condition at these points before continuing.

1.1 Objective

• To implement a coordinate-based slide simulation in C.
• To store slide endpoints using a structure.
• To find a suitable slide below the current position.
• To update position and energy during movement.
• To detect specific blocking conditions using a function.
• To display the final coordinates.

1.2 Overview

The program first reads and stores all slide data. It then reads the starting coordinate and energy. A loop searches for the best slide candidate. If no slide is selected, the vertical coordinate is set to zero and the simulation stops. Otherwise, the program follows the selected slide or continues searching from its lower endpoint.

1.3 Features

• Supports multiple slides using an array.
• Uses a structure to group each slide's coordinates.
• Checks movement blockage by coordinate and direction.
• Decreases energy as the point moves.
• Prints the final x- and y-coordinates.

2. PROGRAM ANALYSIS

The program represents each slide with four integer coordinates: lx, ly, ux, and uy. The global array s stores up to 60 slides, while n records the number of slides entered by the user. The endpoints are rearranged so that the endpoint with the smaller y-coordinate is stored as the lower endpoint.

2.1 Problem Definition

The task is to determine the final coordinate of a point moving through a coordinate-based arrangement of diagonal slides under a given set of rules. The program must identify candidate slides, choose the candidate with the greatest eligible y-coordinate, handle movement along a selected slide, check blocking conditions, and update energy.

2.2 Project Overview

The program uses the Slide structure to represent each slide. The blocked() function checks other slides that begin at a specified lower endpoint and compares their direction with the current direction. The main() function manages input, candidate selection, movement, energy updates, and final output.

2.3 Scope and Limitations

Scope:

- Handles up to 60 slides as declared in the program.
- Uses integer coordinates and an integer energy value.
- Simulates movement and selected blocking rules.
- Prints the final coordinate in a compact format.

Limitations:

- Input validation is not included.
- The program assumes input values follow the expected format.
- It does not draw a graphical view of the slides.
- The blocking rule checks matching lower endpoints and direction only.

3. SYSTEM DESIGN

The system follows an input, processing, and output design. Input includes the slide count, slide endpoints, starting coordinate, and energy. Processing identifies a candidate slide and simulates movement according to the program's conditions. Output displays the final coordinate.

Input Design

The program reads the number of slides n. For every slide, it reads x1, y1, x2, and y2. After reading all slides, it reads bx, by, and e, which represent the starting x-coordinate, starting y-coordinate, and available energy.

Processing Design

The endpoints are stored in the Slide structure in lower-to-upper vertical order. The program searches all slides whose horizontal span contains bx and whose calculated y-coordinate is not above by. It selects the candidate with the highest eligible y-coordinate, with an additional preference when candidates share that y-coordinate.

Movement and Output Design

If the selected point is at the lower endpoint, the program continues its search. Otherwise, it determines a horizontal direction and number of movement steps. During each step, bx changes by one unit in that direction, by decreases by one, and e decreases by one. The final bx and by are printed.

3.1 Modules

1. Input Module: Reads slide data, starting position, and energy.
2. Coordinate Storage Module: Stores normalized endpoints in the Slide array.
3. Blocking Check Module: Uses blocked() to check matching endpoints and directions.
4. Candidate Selection Module: Finds the best eligible slide.
5. Movement Module: Updates coordinates and energy.
6. Output Module: Prints the final coordinates.

4. METHODOLOGY

The project is implemented through a sequence of steps: read input, normalize slide endpoints, identify the next eligible slide, evaluate movement and blocking conditions, update the current state, and display the final coordinates.

4.1 Problem Analysis

The main challenge is selecting the next slide based on the current horizontal and vertical coordinates. A slide is considered only if bx lies between its horizontal endpoints and the calculated slide height does not exceed by. The code then selects the candidate with the greatest eligible height, applying its tie-breaking condition.

4.2 System Design

A Slide structure groups the coordinates of one slide. The global array stores the slides so that each iteration can examine every candidate. The blocked() function isolates the path-blocking check from the main movement logic.

4.3 Program Development

The program is written in C. The stdio.h header provides scanf() and printf() for input and output. The stdlib.h header provides abs() for absolute differences. Arrays, structures, loops, and conditional statements are used to represent and process the slide arrangement.

4.4 Testing and Validation

Testing should include a single slide, multiple slides, a starting position with no eligible slide, and cases where a blocking condition is detected. The resulting final coordinate should be checked against the movement rules implemented in the code. The program assumes valid input, so malformed values are outside the intended test cases.

4.5 Result and Documentation

When the program completes, it prints the final x-coordinate and y-coordinate separated by a space. The report documents the structure, helper function, candidate-selection process, movement rules, energy updates, sample input, and sample output.

4.6 Algorithm

1. Start.
2. Read the number of slides n.
3. Read each slide's two endpoints and store the lower endpoint first.
4. Read starting coordinates bx, by and energy e.
5. Search all slides for an eligible candidate under bx.
6. If no candidate exists, set by to zero and stop the loop.
7. If the selected point is the lower endpoint, repeat the search.
8. Determine direction and movement steps along the slide.
9. Check for a blocking slide and apply the energy condition.
10. Move one step at a time, updating bx, by and e.
11. Print bx and by. Stop.

4.7. FLOWCHART

START
↓
Read n and slide endpoints
↓
Normalize and store endpoints
↓
Read bx, by and energy e
↓
Find best eligible slide
↓
Is a slide found?
No → Set by = 0 and finish
Yes ↓
Is selected point the lower endpoint?
Yes → Search again
No ↓
Check blockage and energy
↓
Move along slide; update bx, by, e
↓
Repeat search
↓
Print bx and by
↓
STOP

5. SOURCE CODE

File name: SlideTraversal.c

Part 1: Header files, data structure, blocking function, and input processing.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct { int lx, ly, ux, uy; } Slide;

static Slide s[60];
static int n;

int blocked(int self, int x, int y, int dx) {
    for (int i = 0; i < n; i++) {
        if (i == self) continue;
        if (s[i].lx == x && s[i].ly == y) {
            int d = (s[i].ux > s[i].lx) ? 1 : -1;
            if (d == dx) return 1;
        }
    }
    return 0;
}

int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        int x1, y1, x2, y2;
        scanf("%d %d %d %d", &x1, &y1, &x2, &y2);
        if (y1 <= y2) {
            s[i].lx = x1; s[i].ly = y1;
            s[i].ux = x2; s[i].uy = y2;
        } else {
            s[i].lx = x2; s[i].ly = y2;
            s[i].ux = x1; s[i].uy = y1;
        }
    }
```

5. SOURCE CODE (CONTINUED)

Part 2: Candidate selection, movement, energy updates, and output.

```
    int bx, by, e;
    scanf("%d %d %d", &bx, &by, &e);

    int done = 0;
    while (!done) {
        int best = -1, besty = -1, bestLow = 1;
        for (int i = 0; i < n; i++) {
            int mn = s[i].lx < s[i].ux ? s[i].lx : s[i].ux;
            int mx = s[i].lx > s[i].ux ? s[i].lx : s[i].ux;
            if (bx < mn || bx > mx) continue;
            int yy = s[i].ly + abs(bx - s[i].lx);
            if (yy > by) continue;
            int isLow = (bx == s[i].lx);
            if (yy == by && isLow) continue;
            if (yy > besty || (yy == besty && bestLow && !isLow)) {
                best = i; besty = yy; bestLow = isLow;
            }
        }

        if (best < 0) { by = 0; break; }
        by = besty;
        if (bestLow) continue;

        int dx = (s[best].lx > bx) ? 1 : -1;
        int steps = abs(s[best].lx - bx);

        if (blocked(best, bx, by, dx)) {
            if (e < bx * by) { done = 1; break; }
            e -= bx * by;
        }
        for (int k = 0; k < steps; k++) {
            if (e == 0) { done = 1; break; }
            bx += dx; by -= 1; e--;
            if (blocked(best, bx, by, dx)) {
                if (e < bx * by) { done = 1; break; }
                e -= bx * by;
            }
        }
    }
    printf("%d %d", bx, by);
    return 0;
}
```

Important variables

• bx, by: current coordinates of the moving point.
• e: remaining energy.
• best: index of the selected slide.
• besty: selected slide's calculated y-coordinate.
• bestLow: indicates whether the point is at the selected slide's lower endpoint.
• dx: horizontal movement direction.

6. INPUT AND OUTPUT

Sample Input

```
1
0 0 3 3
2 2 10
```

Input Explanation

The first value indicates that there is one slide. The slide endpoints are (0, 0) and (3, 3). The starting position is (2, 2), and the initial energy is 10.

Sample Output

```
0 0
```

Output Explanation

The point starts at (2, 2) and is associated with the diagonal slide. It moves toward the lower endpoint, decreasing the y-coordinate and energy at each movement step. After reaching the lower endpoint, the next search finds no eligible slide to continue from, so the program ends with by equal to zero and prints the final coordinate.

Additional Test Ideas

1. No slides: check how the program handles an empty slide list with a valid starting coordinate.
2. Starting position outside all slide spans: verify the no-candidate path.
3. Two slides with a shared lower endpoint: check the blocking function.
4. Low energy: check the condition that stops movement when the remaining energy is insufficient.

7. DISCUSSION AND CONCLUSION

7.1 Discussion

The program combines coordinate processing and conditional movement. The Slide structure keeps the four endpoints together, and the array makes it possible to check each slide during candidate selection. The blocked() function separates one specific path condition from the main logic. The movement loop updates the position and energy until the simulation reaches a stopping condition.

The implementation is a compact simulation rather than a graphical application. It relies on integer input and the exact rules expressed in the supplied code. Because the program does not validate scanf() results or check whether n exceeds the array capacity, valid input within the declared limit is expected.

7.2 Conclusion

The project demonstrates how C structures, arrays, functions, loops, and conditional statements can be combined to simulate movement over diagonal slides. It reads the slide arrangement and initial state, checks candidate slides and blocking conditions, updates coordinates and energy, and displays the final position. The code can be extended with input validation, clearer error handling, and a visual representation of the coordinate plane.

8. REFERENCES

1. Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, Prentice Hall.
2. Stephen G. Kochan, *Programming in C*, Pearson.
3. cppreference.com, C language reference: https://en.cppreference.com/w/c
4. GNU Compiler Collection (GCC) documentation: https://gcc.gnu.org/onlinedocs/